Unreal Engine 5.7

Meta Quest Standalone Deployment

Reference Guide & Settings Checklist

Verified working configuration for UE 5.7 → Quest APK deployment

Created by

Reggie “Eldridge” Felder

Version 1.3

June 6, 2026

PART 1

For Presenters & Testers

If you've been sent an APK to view or present, start here. No developer tools required.

Testing and Presenting VR Packages

If you've received an APK file containing a VR presentation (or any other Quest VR build) and need to view it in a headset, this section is the entire setup. None of the developer tools in Part 2 are required.

Recommended presenter setup (Meta Quest Developer Hub only)

For a fresh computer, this is the entire required install. MQDH bundles its own adb, handles USB drivers, and provides drag-and-drop APK installation. No Visual Studio, no Unreal, no Android Studio, no JDK, no environment variables.

One-time setup

- Install Meta Quest Developer Hub from developers.meta.com/horizon/downloads/package/oculus-developer-hub-win
- Add a Meta Developer organization to your Meta account (free, 2 minutes at developers.meta.com)
- Enable Developer Mode on the headset (Meta Horizon mobile app → Devices → headset → Headset Settings → Developer Mode, or via admin if the headset is managed)

Each time you receive an updated APK

- Once you have the APK file (received via cloud share, USB drive, email, etc.), save it somewhere accessible on the PC
- Plug the headset into PC via USB-C, accept the "Allow USB debugging" prompt in the headset
- Open MQDH — headset auto-detects
- Drag the APK file onto MQDH's Apps tab — installs in 30-60 seconds
- Put on headset → Library → filter to "Installed" (or "Unknown Sources") → click the app to launch

Headset compatibility

The APK runs on any Quest headset listed in its supportedDevices manifest entry — typically Quest Pro and Quest 3, sometimes Quest 2. If the build was packaged for a specific headset, it may refuse to install on others. Check with the developer who sent the APK if unsure.

Alternative for command-line-comfortable users

Standalone Android Platform-Tools (developer.android.com/tools/releases/platform-tools, ~10MB) gives just adb. Install with: adb install -r [path]. Faster than MQDH for batch installs but requires terminal use. MQDH is friendlier for most presenters.

Common issues during install

Symptom	Likely cause / fix
Headset not detected by MQDH	USB cable is charge-only — use a data cable. Or developer mode is off — check the headset's Settings → Advanced → Developer.
INSTALL_FAILED_OLDER_SDK / HZOS version error	Headset firmware is too old for this APK. Update Horizon OS via Settings → System → Software Update (or contact admin if managed).
INSTALL_FAILED_USER_RESTRICTED	Managed device policy blocks sideloading. Contact your IT/admin to enable developer permissions.
APK installed but launches as flat 2D screen	Send back to developer — Meta XR plugin issue in the build, not something you can fix on receiving end.
App doesn't appear in headset library	Filter the library by "Unknown Sources" or "Installed" — sideloaded apps don't appear in the default "All" view.

PART 2

For Developers

If you're building VR packages for Quest from Unreal Engine 5.7, work through the sections below in order.

1. Installation Order Overview

Install order matters — out-of-order installs cause broken paths and missing licenses. Read each step's Purpose before deciding to skip anything.

The order at a glance

- **Step 1:** Install Visual Studio 2022 (provides C++ compiler Unreal needs)
- **Step 2:** Install Unreal Engine 5.7 with Android target platform
- **Step 3:** Install Android Studio + configure SDK Manager (JDK, SDK, NDK, build tools, CMake)
- **Step 4:** Set Windows environment variables (JAVA_HOME, PATH)
- **Step 5:** Run Unreal's SetupAndroid.bat to register everything
- **Step 6:** Install Meta Horizon Link PC software (for VR Preview / tethered testing)
- **Step 7:** Install Meta XR plugin into your Unreal project
- **Step 8 (optional):** Install Meta Quest Developer Hub (GUI for managing headset / inspecting logs)
- **Step 9:** Verify the full installation (JDK, JAVA_HOME, ADB, OpenXR runtime, Unreal tooling)

After Step 9, proceed to Section 2 to configure the project.

Step 1: Install Visual Studio 2022

Purpose

Provides the C++ compiler Unreal needs to build any project, including Blueprint-only ones.

What to install

- Download Visual Studio 2022 Community Edition (free): visualstudio.microsoft.com/vs/community/
- During install, in the Workloads tab, check "Game Development with C++"
- Optional: Also check ".NET desktop development" — Epic's tools occasionally use .NET utilities
- Click Install (download is 5-15 GB depending on options)

Step 2: Install Unreal Engine 5.7

Purpose

The engine you'll build in. Install it before Android Studio so its SetupAndroid.bat (Step 5) exists when needed.

What to install

- Install the Epic Games Launcher: store.epicgames.com/en-US/download
- In the Launcher: Library → click the + next to Engine Versions → select 5.7 → Install
- CRITICAL: Click the Options dropdown before installing. Under Target Platforms, ensure ANDROID is checked.
- Without the Android target platform option, Unreal cannot build APKs at all — even with all other tooling installed correctly. This is the most commonly missed step.
- Optional but recommended: also check Engine Source and Editor symbols for debugging
- Click Install (download is 30-60 GB depending on options)

Step 3: Install Android Studio + Configure SDK Manager

Purpose

Acts as the package manager for Google's Android components (SDK, NDK, build tools, CMake) — Unreal doesn't bundle these. You'll never use Android Studio as an IDE; Unreal handles all building. Bonus: it ships with JDK 21 (JBR), so you may not need a separate JDK install.

Install Android Studio

- Download from developer.android.com/studio
- Run the installer, accept defaults
- On first launch, decline any sample project setup — you don't need it

Configure the SDK Manager

From the Android Studio Welcome screen, click "More Actions" → "SDK Manager". (Alternative: if you have a project already open, use File → Settings → Languages & Frameworks → Android SDK.)

SDK Platforms tab

- Check Android 14 (API Level 34)

SDK Tools tab

Check the box "Show Package Details" at the bottom-right of the panel — this lets you select specific versions instead of "latest."

Required: check these

Component	Version / Notes

Android SDK Build-Tools	35.0.1 — compiles Android resources into APK format
NDK (Side by side)	27.2.12479018 — native toolchain for compiling C++ to ARM64
Android SDK Platform-Tools	Latest — includes adb (the bridge to your headset)
Android SDK Command-line Tools	Latest — modern sdkmanager and license utilities
CMake	3.22.1 — build orchestration for native code

Click Apply. Android Studio downloads ~2 GB of components.

Already installed automatically (leave alone)

These are system utilities Android Studio manages on its own. They appear checked by default; don't uncheck them.

Component	Why it's there
SDK Patch Applier v4	Lets Android Studio apply small patch updates to SDK components instead of full redownloads. Tiny utility, system-managed.
Older CMake versions (e.g., 3.10.2)	May appear if previously installed. Multiple CMake versions can coexist harmlessly. No need to uninstall older ones.

Do NOT check these (irrelevant to Quest/VR development)

The general rule: anything mentioning "Auto," "Emulator," "Google Play," "Web," "Layout Inspector," or "Support Repository" is for Android phone, tablet, or automotive development and is irrelevant to Quest VR.

Component	What it actually is
Android Auto API Simulators	For car infotainment app development
Android Auto Desktop Head Unit Emulator	Emulates a car dashboard for testing — not useful for Quest
Android Emulator	Virtual Android phones for testing — Quest is the target, not emulators. Even if you later add Android phone development to your workflow, a physical phone is a better test target than the emulator. Only install if you specifically need to test on Android versions or hardware you don't physically own.
Android Emulator Hypervisor Driver (Intel/AMD)	Hardware acceleration for the Android Emulator only — irrelevant for Quest
Android Support Repository	Deprecated; replaced by Google's Maven repository years ago

Google Play APK Expansion library	For Play Store apps over 100MB — Quest uses sideloading, not Play Store
Google Play Instant Development SDK	For "instant apps" (no-install Android apps) — Quest doesn't support this
Google Play Licensing Library	DRM for paid Play Store apps — irrelevant for Quest
Google Play services	Phone-specific (achievements, leaderboards, in-app purchases on Android phones)
Google Repository	Deprecated artifact storage; replaced by Maven
Google USB Driver	Generic Android USB drivers. Quest uses its own driver via Meta Horizon Link installer. Install only if adb devices can't find your headset after Meta Horizon Link is installed.
Google Web Driver	Browser test automation — not for Quest or any standalone app
Layout Inspector image server (any version)	Phone UI debugging tool — irrelevant for Quest

Click Apply. Android Studio downloads ~2 GB of components.

JDK 21 source options

⚠ IMPORTANT — You should already have Option 1 installed. Android Studio (Step 3 above) bundles its own JDK 21 — there is no separate download or install action needed. Do NOT install all three options. Pick exactly ONE. Reading the descriptions below before acting will save time.

UE 5.7 requires JDK 21. Three valid sources, all functionally identical:

Option 1: Android Studio's bundled JBR (recommended)

Already installed in Step 3. JetBrains Runtime is OpenJDK 21 in current Android Studio versions.

Property	Value
Install location	`C:\Program Files\Android\Android Studio\jbr`
JAVA_HOME value	(same as install location)
Pros / Cons	Already on disk, no extra download. JDK updates only when Android Studio updates.

Option 2: Microsoft Build of OpenJDK

Microsoft-signed OpenJDK distribution.

Property	Value
Download	learn.microsoft.com/en-us/java/openjdk/download

Choose	JDK 21 LTS, Windows x64, MSI installer
Install location	`C:\Program Files\Microsoft\jdk-21.0.x.x-hotspot`
Tip	Check "Set JAVA_HOME variable" during install to skip Step 4 manual setup

Option 3: Eclipse Temurin

Community-maintained OpenJDK (Adoptium working group).

Property	Value
Download	adoptium.net/temurin/releases/?version=21
Choose	JDK 21 (LTS), Windows x64, .msi installer
Install location	`C:\Program Files\Eclipse Adoptium\jdk-21.x.x.x-hotspot`
Tip	Check "Set JAVA_HOME variable" during install to skip Step 4 manual setup

Cannot get JDK from

- Meta XR Plugin — does not bundle a JDK; assumes JAVA_HOME is configured
- Epic Games Launcher — does not include a standalone JDK

Step 4: Set Windows Environment Variables

Purpose

Build tools (SetupAndroid.bat, Gradle, ADB) locate Java via JAVA_HOME and PATH. Without them, SetupAndroid fails with cryptic errors. Skip this step if your JDK installer's "Set JAVA_HOME" checkbox handled it (verify in Step C). Android Studio's JBR does NOT auto-set these — Option 1 users must do this manually.

Open the Environment Variables dialog

- Press the Windows key, type "environment variables"
- Click "Edit the system environment variables" — opens System Properties
- In System Properties, click the "Environment Variables..." button at the bottom

Two sections appear: "User variables" (top) and "System variables" (bottom). Use User variables — applies just to your account, no admin rights needed.

Step A: Create or update JAVA_HOME

First, check the User variables list for an existing JAVA_HOME entry — it may already exist from a JDK installer or previous Java install.

If JAVA_HOME does NOT exist (most common — Option 1 / Android Studio JBR users)

- In the User variables section, click "New..."
- Variable name: JAVA_HOME
- Variable value: the install path of your chosen JDK (e.g., C:\Program Files\Android\Android Studio\jbr)
- Click OK

If JAVA_HOME ALREADY exists — do NOT create a new one

Edit the existing entry instead — Windows allows only one variable per name:

- Double-click the existing JAVA_HOME entry
- Confirm value points at a JDK 21 install (not JDK 8, 11, or 17)
- If correct, leave it and skip to Step B. If older, replace with JDK 21 path

Tip: Open File Explorer first and confirm the path actually exists. Typos are a common cause of failed builds.

Step B: Add %JAVA_HOME%\bin to PATH (edit existing variable)

PATH already exists — do NOT create a new one. Edit it:

- Double-click the User Path entry
- Click "New" and type: %JAVA_HOME%\bin (include percent signs)
- Move it to the top, or at minimum above any other Java entries (look for "Java", "jdk", "jre", "Oracle")
- Click OK on each dialog

Why use %JAVA_HOME%\bin instead of the full path? Windows expands the variable at runtime — if you upgrade JDK versions later, PATH auto-follows JAVA_HOME.

Verify the changes

Close any existing terminals (they keep old PATH) and open a fresh Command Prompt:

```
echo %JAVA_HOME%
```

Should print your JDK install path.

```
java -version
```

Should report version 21 — NOT 8, 17, or "command not recognized."

If java -version reports the wrong version

Another Java install is winning on PATH. Three common causes:

Cause 1: System PATH is checked BEFORE User PATH

Putting %JAVA_HOME%\bin at the top of User PATH does NOT override System PATH. Run:

```
where java
```

This shows all java.exe locations in priority order. The first line is what Windows actually uses.

Cause 2: Oracle Java's javapath is in System PATH

Oracle's installer adds C:\Program Files\Common Files\Oracle\Java\javapath to System PATH. It persists even after Oracle Java is uninstalled, or gets silently installed by other software (Adobe, certain games).

Fix: Environment Variables → System variables → Path → delete that entry. Optionally uninstall Oracle Java via Settings → Apps.

Cause 3: Uninstalling Oracle Java may wipe %JAVA_HOME%\bin from PATH

Oracle's uninstaller sometimes removes other Java-related PATH entries. Symptom: java -version reports "command not recognized" after uninstalling Oracle. Fix: re-add %JAVA_HOME%\bin per Step B. Always re-verify PATH after installing or uninstalling any Java.

Alternative: add to System PATH instead

If problems persist, add %JAVA_HOME%\bin to System PATH (same edit process, System variables section). System PATH applies machine-wide and is checked first.

Step 5: Run Unreal's SetupAndroid Script

Purpose

Android Studio installs SDK components on disk; SetupAndroid registers them with Unreal. The script auto-accepts SDK licenses non-interactively, writes paths to Unreal's config (so Project Settings auto-populate), and configures keystore and Gradle files. It does NOT re-download components — Step 3 must run first.

Run the script

Run as Administrator from this exact location:

```
[UE_Install_Folder]\Engine\Extras\Android\SetupAndroid.bat
```

Example: D:\Program Files\Epic Games\UE_5.7\Engine\Extras\Android\SetupAndroid.bat

Common error: javax.xml.bind ClassNotFoundException

JAVA_HOME points at JDK 8 (which has JAXB) or no JDK at all. Confirm JAVA_HOME points to JDK 21 and %JAVA_HOME%\bin is at the top of PATH (Step 4), then re-run.

Step 6: Install Meta Horizon Link PC Software

Purpose

Enables tethered PC VR streaming (used by Unreal's VR Preview), provides the Meta OpenXR runtime, and supports wireless Air Link. Without it, every test iteration requires packaging the full APK (5-30 min). With it, hit Play and instantly test in VR — essential for productive development.

Install

- Download from meta.com/quest/setup/
- Choose "Meta Horizon Link for PC"

- Run installer, sign in with the same Meta account that's on the headset
- After install: open the app, Settings → General → click "Set Meta Horizon Link as active OpenXR Runtime"

Step 7: Install Meta XR Plugin into Your Unreal Project

Purpose

Bridges Unreal's generic OpenXR with Quest-specific features (passthrough, hand tracking, dynamic foveated rendering, eye tracking, Application SpaceWarp). Also provides Quest-specific Project Settings, the Meta XR Setup Tool, and the supportedDevices manifest entry. Without it, your APK may launch as a flat 2D screen instead of immersive VR. Do not skip.

Where to download

Direct download from Meta (recommended — Fab marketplace version sometimes lags):

```
https://developers.meta.com/horizon/downloads/package/unreal-engine-5-integration
```

Sign in with your Meta Developer account (free at developers.meta.com if you don't have one).

How to install

- Close Unreal Editor
- Extract the downloaded zip
- Inside, find the MetaXR folder
- Copy MetaXR into either: [YourProject]/Plugins/ (per-project) OR [UE_Install]\Engine\Plugins\Marketplace\ (engine-wide)
- Reopen project; allow rebuild if prompted
- Edit → Plugins → search "Meta XR" → confirm enabled
- Restart editor when prompted

Step 8: Install Meta Quest Developer Hub (Optional)

Purpose

Optional GUI wrapper for ADB. Useful for: drag-and-drop APK installs (good for non-technical testers), real-time headset log inspection (helps debug launch crashes), and mirroring the headset's display to your PC. Solo developers comfortable with command-line ADB can skip this step.

Install

- Download from developers.meta.com/horizon/downloads/package/oculus-developer-hub-win
- Run installer
- On first launch, sign in with your Meta Developer account
- Connect headset via USB; MQDH detects it automatically once Developer Mode is on (see Headset Setup section)

Step 9: Verify Installation

Purpose

Final check before configuring your project. Catches problems while they're cheap to fix.

Run these commands in a fresh Command Prompt

```
java -version
```

Expect: "openjdk version 21.0.x". If you see a different version, JAVA_HOME or PATH is wrong.

```
echo %JAVA_HOME%
```

Expect: your JDK install path. If empty, JAVA_HOME isn't set.

```
adb version
```

Expect: a version number like "Android Debug Bridge version 1.0.41". If you see "command not recognized," platform-tools isn't on PATH — add C:\Users\[you]\AppData\Local\Android\Sdk\platform-tools to PATH.

```
reg query "HKLM\SOFTWARE\Khronos\OpenXR\1" /v ActiveRuntime
```

Expect: a path containing "oculus_openxr_64.json". If the key doesn't exist or shows SteamVR, open Meta Horizon Link → Settings → General → set OpenXR runtime.

Verify Unreal sees the Android tooling

- Open Unreal 5.7
- Open or create a project
- Edit → Project Settings → Platforms → Android SDK
- All five fields (SDK, NDK, JAVA, SDK API Level, NDK API Level) should be auto-populated. If any are blank, SetupAndroid.bat didn't run successfully — go back to Step 5.

Once everything verifies, you're ready to configure the project (next section).

2. Configure Your Unreal Project

With the toolchain installed, configure project-specific settings. All paths below are at Edit → Project Settings.

Platforms → Android SDK

Setting	Value
Location of Android SDK	C:\Users\[username]\AppData\Local\Android\Sdk
Location of Android NDK	C:\Users\[username]\AppData\Local\Android\Sdk\ndk\27.2.12479018
Location of JAVA	Match your JAVA_HOME path from Step 3 (Android Studio JBR, Microsoft, or Eclipse Temurin)

SDK API Level	android-34
NDK API Level	android-33 (Meta XR currently recommends android-32 to android-33; must be ≤ SDK API Level)

Important: NDK version vs NDK API Level

These two settings sound similar but mean different things — confusing them is a common build error source:

- **NDK version** (e.g., 27.2.12479018) — the toolchain bundle on disk. Set by what you install in Android Studio.
- **NDK API Level** (e.g., android-33) — the minimum Android OS API that compiled code targets at runtime.

Meta XR Setup Tool warnings about NDK "between android-32 and android-33" refer to NDK API Level. Rule of thumb: NDK API Level ≤ SDK API Level.

Platforms → Android

If you see a "Configure Now" yellow banner, click it once. The page should turn to a green "Platform files are writeable" banner.

APK Packaging section

Setting	Value
Android Package Name	com.[yourcompany].[yourproject] (must be unique, lowercase, no spaces)
Application Display Name	Whatever you want to appear under the app icon in the headset library
Minimum SDK Version	34
Target SDK Version	34
Package game data inside .apk?	Checked (single-file APK, easier to distribute)
Support arm64	Checked
Support armv7	Unchecked (Quest is 64-bit only)

Advanced APK Packaging section

Verify this entry exists under "Extra Settings for <application> section":

```
<meta-data android:name="com.oculus.supportedDevices"
android:value="quest2|questpro|quest3" />
```

Build / Distribution Signing

For internal testing and sideloading: leave the distribution signing fields blank. Unreal automatically debug-signs the APK, which is sufficient for sideloaded installs.

Plugins → Meta XR

These settings live under Edit → Project Settings → Plugins → Meta XR (visible only after the Meta XR plugin is installed and enabled).

General section

Setting	Value
XR API	Epic Native OpenXR (Recommended)
Color Space	P3 (Recommended)
Minimum Horizon OS Version	Set to lowest your headset firmware supports — important if testing on older Horizon OS
Target Horizon OS Version	Match a known-supported version, not "Latest" if your headset firmware is old
Horizon OSVersion Override	Unchecked

Mobile section

Setting	Value
Supported Meta Quest devices	Array containing Quest Pro, Quest 3 (and Quest 2 if needed)
Suggested CPU Perf Level	SustainedLow
Suggested GPU Perf Level	SustainedHigh
Hand Tracking Support	Controllers Only (unless your scene actually uses hand tracking)

After configuring Meta XR settings, run the Project Setup Tool: Project Settings → Plugins → Meta XR → Launch Meta XR Project Setup Tool. Apply every red error fix it suggests; review yellow warnings.

Engine → Rendering (critical for Quest performance)

Setting	Value
Mobile HDR	OFF (critical — leaving on kills performance and breaks MSAA)
Forward Shading	ON
Mobile Multi-View	ON
Mobile Anti-Aliasing Method	MSAA
MSAA Sample Count	4x

Dynamic Global Illumination Method	None (Lumen is not supported on Quest standalone)
Reflection Method	None or Screen Space (no Lumen reflections on Quest)
Auto Exposure	Off, or Manual with fixed value

Project → Description

Setting	Value
Start in VR	Checked

3. Quest Headset Setup

Enable Developer Mode

- **Consumer (unmanaged) headsets:** Meta Horizon mobile app → Devices → headset → Headset Settings → Developer Mode → toggle on. Reboot headset.
- **Enterprise (managed) headsets:** Phone pairing is blocked. IT/admin must enable developer permissions via Meta Admin Center (or third-party MDM like Intune, ArborXR, ManageXR). Once enabled, the toggle appears at: Settings → System → Advanced → Developer.

Connect to PC

- Use a quality USB-C data cable (charge-only cables won't work)
- Plug headset into PC, put on headset
- Accept the "Allow USB debugging?" prompt — check "Always allow from this computer"

Verify connection

In Command Prompt:

```
adb devices
```

Expected output: <serial number> device

If you see "unauthorized", you missed the USB debug prompt — replug or check headset.

If empty list, developer mode is off OR (on managed devices) the policy blocks ADB.

4. Build and Deploy

Package the APK

- Unreal toolbar: Platforms → Android → Package Project
- Flavor: Android (ASTC) — Quest's native texture format
- Build Configuration: Development (good for testing; switch to Shipping only for final builds)
- Choose output folder (e.g., D:\UE_Meta\Android_ASTC\)

- First package: 15–60 minutes (Gradle download + shader compilation). Subsequent packages much faster.

Output APK location:

```
[OutputFolder]\YourProjectName-arm64.apk
```

Install to headset

First install:

```
adb install "D:\path\to\YourProject-arm64.apk"
```

Reinstall (after rebuild):

```
adb install -r "D:\path\to\YourProject-arm64.apk"
```

Uninstall:

```
adb uninstall com.yourcompany.yourproject
```

Launch on headset

Manual launch:

- Press Meta button → Library
- Filter dropdown → Installed (or Unknown Sources on older Horizon OS)
- Click your app to launch

Direct launch from PC (faster for iteration):

```
adb shell am start -n com.yourcompany.yourproject/com.epicgames.unreal.GameActivity
```

Find the exact package name:

```
adb shell pm list packages | findstr -i [partofname]
```

5. Common Errors and Fixes

Error	Cause and Fix
Platform Android is not a valid platform to build	Wrong JDK/NDK/SDK versions for UE 5.7. Verify JDK 21, NDK 27.2.12479018, SDK API 34, NDK API 33.
javax.xml.bind ClassNotFoundException (in SetupAndroid.bat)	JAVA_HOME points at JDK 9+ which lacks JAXB. Install JDK 21, set JAVA_HOME to it.
INSTALL_FAILED_OLDER_SDK / Requires newer HZOS SDK	Headset firmware is older than what your APK targets. Lower Meta XR plugin's Min Horizon OS Version, or update headset firmware via admin.
INSTALL_FAILED_ALREADY_EXISTS	App already installed. Use "adb install -r" to reinstall over existing.
INSTALL_FAILED_USER_RESTRICTED	Managed device policy blocks sideloading. Contact admin.

adb devices empty	Developer Mode off, USB cable charge-only, or admin policy blocks ADB.
APK launches as flat 2D, not VR	Meta XR plugin not enabled, or "Start in VR" unchecked.
Black screen on launch	Mobile HDR still on, or unsupported dynamic light. Check rendering settings.
Frame rate unusable	Lumen, Nanite, or dynamic lighting still active. Bake lighting and disable Lumen/Nanite.

6. VR Preview (Tethered In-Editor Testing)

Runs your scene tethered using your PC's GPU — no packaging, no install. Best for iterating on scene content, blueprints, and materials.

How VR Preview differs from the APK build

Aspect	VR Preview vs Standalone APK
Renderer	Uses PC GPU (much more powerful) vs Quest's mobile GPU
Speed to test	Instant — just click Play vs 2–10 min package + install
Tethered?	Yes (USB or Air Link) vs No, fully untethered
Performance representative?	No — looks/runs better than reality vs Yes, real performance
Mobile renderer issues visible?	No — uses desktop renderer vs Yes, exposes mobile-only bugs
Best used for	Inner loop iteration vs Final validation and sharing

Setup steps (one-time)

Stage 1: Install Meta Horizon Link PC software

Download from meta.com/quest/setup/, choose "Meta Horizon Link for PC," sign in with your Meta account.

Stage 2: Set Meta as the active OpenXR runtime

Without this, VR Preview stays greyed out. In Meta Horizon Link: Settings (gear) → General → OpenXR Runtime → click "Set Meta Horizon Link as active OpenXR Runtime."

If the button is greyed, Meta is likely already active. Verify:

```
reg query "HKLM\SOFTWARE\Khronos\OpenXR\1" /v ActiveRuntime
```

Expected output (Meta is active):

```
ActiveRuntime    REG_SZ    C:\Program
Files\Oculus\Support\oculus-runtime\oculus_openxr_64.json
```

If a different runtime shows: close Meta Horizon Link, restart as Administrator, retry. If SteamVR keeps reclaiming: SteamVR → Settings → Developer → uncheck "Set SteamVR as OpenXR runtime," then re-set Meta. Last resort: manually edit HKLM\SOFTWARE\Khronos\OpenXR\1 → ActiveRuntime via regedit.

Stage 3: Verify Unreal plugins

In Unreal: Edit → Plugins

- Confirm OpenXR plugin is enabled (default)
- Confirm Meta XR plugin is enabled (installed earlier)

Per-session: connect headset to PC (sequence matters)

CRITICAL: The headset must be in the Link environment BEFORE launching Unreal. The editor checks for an active OpenXR session at startup — if Unreal is open when you connect Link, VR Preview stays greyed until you restart the editor.

Correct order

- **1.** Close Unreal if open
- **2.** Start Meta Horizon Link on PC (verify running in system tray)
- **3.** Plug in headset via USB-C (or use Air Link)
- **4.** Put on headset, engage Link (see below)
- **5.** Verify Link dash environment is visible (not Horizon OS home)
- **6.** Launch Unreal — VR Preview will detect the active session

Engaging Link in the headset

If Link doesn't auto-launch: Meta button → clock/time (Quick Settings) → "Link" tile → "Open Dash" or "Launch Link." For Air Link: Quick Settings → Link → switch to Air Link mode → select your PC. The headset switches to the Link dash with PC mirroring.

Confirming Link is active

You should see: a virtual lounge with floating PC monitor, your PC desktop mirrored, or the Link dashboard. If you see the regular Horizon OS home (Library, Store, friends), Link is NOT active — re-engage via Quick Settings → Link → Open Dash.

Launching VR Preview

- Restart Unreal if it was open before you set the OpenXR runtime — VR Preview detection happens at editor startup
- Click the dropdown arrow next to the Play button in the toolbar
- VR Preview should now be available (not greyed out)
- Click VR Preview — your scene appears in the headset within a few seconds

Common VR Preview issues

Symptom	Cause and Fix
VR Preview greyed out — most common cause	Unreal was launched BEFORE the headset was in the Meta Horizon Link app mode. Close Unreal, engage Link in headset (Quick Settings → Link → Open Dash), confirm you see the Link dash environment, THEN launch Unreal
VR Preview greyed + "Set as active OpenXR runtime" button also greyed in the Meta Horizon Link app	Meta is likely already the active runtime — verify with: reg query "HKLM\SOFTWARE\Khronos\OpenXR\1" /v ActiveRuntime — should show oculus_openxr_64.json. If correct, the issue is the connection sequence above, not OpenXR
Greyed out + SteamVR installed	SteamVR is intercepting OpenXR. Open SteamVR → Settings → Developer → uncheck "Set SteamVR as OpenXR runtime," then re-set Meta as active
VR Preview clickable but headset shows nothing	Headset isn't actively in Link mode — confirm you see the Link dash environment, not Horizon OS home
Scene launches on monitor instead of headset	Headset is not in Link mode. Re-engage Link before clicking VR Preview
OpenXR runtime button greyed in the Meta Horizon Link app	Either Meta is already active (verify in registry), or the app needs admin rights — close, restart as Administrator
Tracking is jittery or laggy	Air Link bandwidth issue — switch to wired USB. Or close background apps competing for GPU/network
Editor freezes when clicking VR Preview	Lumen or Nanite still on globally. Project Settings → Engine → Rendering → confirm both are off project-wide, not just for Android

Important caveat

VR Preview uses your PC GPU, so scenes look/run better than they will on standalone hardware. Mobile renderer bugs and performance issues may only appear in the APK build. Use VR Preview for fast iteration, but periodically validate with a real APK.

7. Iteration Workflow (after first deploy)

Two loops, used by purpose:

Inner loop — VR Preview (instant)

For scene composition, blueprints, materials, lighting:

- **1.** Engage Link FIRST: Quick Settings → Link → Open Dash
- **2.** Launch Unreal (order matters)
- **3.** Edit, then click Play dropdown → VR Preview

Outer loop — standalone APK (2–10 min)

For performance validation, mobile renderer testing, sharing builds:

- **1.** Edit scene
- **2.** Platforms → Android → Package Project
- **3.** adb install -r [path-to-apk]
- **4.** adb shell am start -n [package]/com.epicgames.unreal.GameActivity
- **5.** Test; unplug USB to verify untethered

Recommended pattern

Iterate in VR Preview daily. Validate with full APK at milestones (new mechanic, level complete, before sharing). Catches mobile-specific issues without slowing inner-loop work.

8. Reference Links

- Meta XR Plugin download: developers.meta.com/horizon/downloads/package/unreal-engine-5-integration
- Meta Horizon Link setup: meta.com/quest/setup/
- Meta Quest Developer Hub: developers.meta.com/horizon/downloads/package/oculus-developer-hub-win
- UE Android setup docs: dev.epicgames.com/documentation/en-us/unreal-engine/setting-up-unreal-engine-projects-for-android-development
- Horizon OS minimum versions reference: developers.meta.com/horizon/documentation/unity/min-os-versions/
- Meta Admin Center (for managed devices): work.meta.com
- JDK 21 - Microsoft Build of OpenJDK: learn.microsoft.com/en-us/java/openjdk/download
- JDK 21 - Eclipse Temurin: adoptium.net/temurin/releases/?version=21
- JDK 21 - Android Studio JBR: bundled with Android Studio; no separate download needed